
VEX V5 Competition Drivetrain Development Guide

From chassis design to competition-ready code

A comprehensive guide for new VEX V5 builders

Table of Contents

1. [Drivetrain Types](#drivetrain-types)
 2. [Chassis Design Fundamentals](#chassis-design-fundamentals)
 3. [Motor Configuration](#motor-configuration)
 4. [Coding Your Drivetrain](#coding-your-drivetrain)
 5. [Common Issues & Fixes](#common-issues--fixes)
 6. [Competition Tips](#competition-tips)
-

Drivetrain Types

1. Tank Drive (H-Drive / Skid Steer)

- Simplest to build and code
- Two sets of wheels: left and right
- Steering = difference in speed between sides
- Best for beginners

2. Holonomic Drive (X-Drive / Mecanum)

- Can move in any direction without turning
- Uses omni-directional wheels at 45-degree angles
- More complex to code but very maneuverable
- Great for competitions requiring quick positioning

3. West Coast Drop Center

- Lower center of gravity
- Fast and compact
- Popular in high-level competition

Recommendation for Beginners

Start with a Tank Drive - it teaches you all the fundamentals before moving to holonomic systems.

Chassis Design Fundamentals

Dimensions

- Maximum starting size: 18" x 18" (457mm x 457mm)
- At match end: Can expand to any size (rule SG4)
- Plan your expansion strategy around game elements

Structural Tips

- Use C-channels and brackets for the frame backbone
- Keep it square - use triangle brackets at corners for rigidity
- Gear ratios matter! See motor configuration below
- Consider weight distribution - center of mass should be low and centered

Wheel Spacing

- Place wheels close to the edges for stability
- Ensure no wheel rubbing against standoffs or crossbars

Motor Configuration

VEX V5 Smart Motors

Motor	RPM	Use Case
100 RPM	100	Torque-heavy tasks (arms, lifts)
200 RPM	200	**Best for standard drivetrains**
600 RPM	600	Speed-heavy (fast drivetrains, shooters)

Recommended Gear Ratios

- 200 RPM motors with 1:1 internal gearing -> all-around balanced drivetrain
- 600 RPM with external 3:1 reduction (~200 RPM effective) -> fast but controlled

Motor Group

In VEXcode, group your motors:

- Left side motors -> one motor group
- Right side motors -> one motor group

Coding Your Drivetrain

Setting Up (VEXcode V5 Text)

Basic Motor Configuration

```
#include "vex.h"
using namespace vex;

// Brain and Controller
brain Brain;
controller Controller1 = controller(primary);

// Left drivetrain motors
motor leftMotor1 = motor(PORT1, ratio18_1, false);
motor leftMotor2 = motor(PORT2, ratio18_1, false);
motor_group leftDrive = motor_group(leftMotor1, leftMotor2);

// Right drivetrain motors
motor rightMotor1 = motor(PORT10, ratio18_1, true); // reversed
motor rightMotor2 = motor(PORT9, ratio18_1, true); // reversed
motor_group rightDrive = motor_group(rightMotor1, rightMotor2);
```

Tank Drive Control

```
// In your main loop (driver control)
while (true) {
    // Get joystick values (-100 to 100)
    int leftSpeed = Controller1.Axis3.position();
    int rightSpeed = Controller1.Axis2.position();

    // Spin motors
    leftDrive.spin(fwd, leftSpeed, pct);
    rightDrive.spin(fwd, rightSpeed, pct);

    wait(20, msec);
}
```

Arcade Drive Control (Alternative)

```
while (true) {  
    int forward = Controller1.Axis3.position(); // forward/back  
    int turn = Controller1.Axis1.position();    // rotation  
  
    int leftSpeed = forward + turn;  
    int rightSpeed = forward - turn;  
  
    // Clamp values to -100..100  
    leftSpeed = (leftSpeed > 100) ? 100 : (leftSpeed < -100) ? -100 : leftSpeed;  
    rightSpeed = (rightSpeed > 100) ? 100 : (rightSpeed < -100) ? -100 : rightSpeed;  
  
    leftDrive.spin(fwd, leftSpeed, pct);  
    rightDrive.spin(fwd, rightSpeed, pct);  
  
    wait(20, msec);  
}
```

Dead Zone (CRITICAL for new builders!)

```
int deadZone = 5; // ignore small joystick values  
  
int applyDeadZone(int value) {  
    if (abs(value) < deadZone) return 0;  
    return value;  
}  
  
// Use it:  
int leftSpeed = applyDeadZone(Controller1.Axis3.position());
```

Adding Autonomous Movement

```
// Move forward a set distance (inches)
void driveForward(float inches, int speed = 75) {
    // VEX 393 motor + 4" wheels: 360 deg = ~12.56 inches
    float degrees = (inches / (4.0 * M_PI)) * 360.0;

    leftDrive.spinFor(fwd, degrees, deg, false);
    rightDrive.spinFor(fwd, degrees, deg, true);
}

// Turn to a specific heading (degrees)
void turnRight(float degrees, int speed = 50) {
    // Tune this value for your robot
    float turnFactor = 0.65;
    float motorDegrees = degrees * turnFactor;

    leftDrive.spinFor(fwd, motorDegrees, deg, false);
    rightDrive.spinFor(reverse, motorDegrees, deg, true);
}

// Autonomous routine
void autonomous(void) {
    driveForward(24);
    turnRight(90);
    driveForward(18);
}
```

Adding PID Control (Intermediate)

```
// Simple driving PID
void drivePID(float targetInches, int maxSpeed = 80) {
    leftDrive.resetPosition();
    rightDrive.resetPosition();

    float kP = 0.5;
    float kD = 0.1;

    float error = targetInches;
    float lastError = 0;

    while (abs(error) > 0.5) {
        float avgDeg = (leftDrive.position(deg) + rightDrive.position(deg)) / 2.0;
        float avgInches = (avgDeg / 360.0) * (4.0 * M_PI);

        error = targetInches - avgInches;
        float derivative = error - lastError;

        float output = (kP * error) + (kD * derivative);
        output = (output > maxSpeed) ? maxSpeed : (output < -maxSpeed) ? -maxSpeed : output;

        leftDrive.spin(fwd, output, pct);
        rightDrive.spin(fwd, output, pct);

        lastError = error;
        wait(20, msec);
    }

    leftDrive.stop(brake);
    rightDrive.stop(brake);
}
```

Common Issues & Fixes

Problem: "My robot drifts to one side"

- Cause: Motors aren't perfectly matched or weight distribution is off
- Fix: Add a correction multiplier, or use a gyro sensor to straighten

```
// Gyro-based straight correction
inertial gyro = inertial(PORT3);

void driveStraight(int speed) {
    float correction = gyro.heading() * 2.0;
    leftDrive.spin(fwd, speed + correction, pct);
    rightDrive.spin(fwd, speed - correction, pct);
}
```

Problem: "My robot slams into walls / overshoots"

- Cause: No braking, moving too fast

- Fix: Use brake brake type and ramp up speed gradually

```
// In main():  
leftDrive.setStopping(coast); // or brake, or hold  
rightDrive.setStopping(coast);
```

Problem: "Autonomous doesn't go far enough"

- Cause: Wheel slip, incorrect math constants
- Fix: Add a fudge factor and test/recalibrate

```
// Instead of theoretical value, measure actual:  
// Drive 100 revolutions, measure distance, calculate FUDGE  
float FUDGE = 1.12; // adjust based on testing  
float degrees = (inches * FUDGE / (wheelDiameter * M_PI)) * 360.0;
```

Problem: "Joystick values jitter when centered"

- Cause: Analog stick noise
- Fix: Always apply a dead zone (see code above) -- this is essential!

Problem: "Robot is too slow / too fast"

- Fix: Change motor cartridge (100/200/600 RPM) or adjust external gear/sprocket ratios
-

Competition Tips

Before the Match

1. ALWAYS test your autonomous on the actual competition field layout
2. Check your radio connection -- ensure Brain and Controller are paired
3. Battery charge! -- Start every match at 100%

During Development

4. Code in small chunks -- get drive working first, then autonomous, then add sensor feedback
5. Use competition switch/field control to simulate real conditions
6. Keep a backup of your last working code version
7. Tune PID constants on carpet -- competition fields are carpet!

Software Best Practices

- Use motor groups so all left/right motors move together
- Always set a stopping mode (brake vs coast)
- Use `setVelocity()` to limit max speed
- Add safety features: motor timeout, stall detection

Recommended Build Order

1. Frame + standoffs
2. Motors + wheels mounted
3. Wire motors to brain -> PORT 1-10
4. Write basic tank/arcade drive -> test!
5. Add dead zone
6. Write basic autonomous (forward + turn)
7. Add sensors (gyro, distance, rotation)
8. Refine autonomous with PID
9. Test, test, test!

Quick Reference Card

VEXcode Shortcut	What It Does
<code>`motor.spin(fwd, 50, pct)`</code>	Spin forward at 50%
<code>`motor.stop(brake)`</code>	Active braking
<code>`motor_group.spinFor(...)`</code>	Drive precise distance
<code>`motor.position(deg)`</code>	Get current encoder value
<code>`motor.resetPosition()`</code>	Zero the encoder
<code>`motor.setStopping(mode)`</code>	coast / brake / hold

Good luck at competition! Remember: drivetrain is the foundation of your robot -- spend time getting this right before adding mechanisms. A fast mechanism on a wobbly drivetrain loses every time.